Python OOP Practice Assignment

Beginner practice workbook | Module 6

Audience: Beginner students. **Goal:** Practise classes, objects, OOP concepts, conditions, loops, functions, and error handling through small tasks and one final project.

How to work

- Create a separate `.py` file for each task, or keep each task clearly separated in one file.
- Read the whole task before coding. Give your classes, methods, and variables clear names.
- Use simple input prompts and print readable results.
- Do not put every action in one large method. Give each method one clear job.
- Some tasks ask you to explain your code. Write the explanation as comments or in your own notes.

Part 1 - Small OOP tasks

Complete these first. Each task focuses on one or two ideas.

Task 1: Class, object, attributes, and methods

Create a `Student` class.

1. Its `__init__` method should receive a student's name, course, and age.
2. Save these values as object attributes using `self`.
3. Add a method called `introduce()` that returns a sentence with the student's name and course.
4. Create two different student objects and print both introductions.
5. Add a class attribute `school_name` that both students share. Add an instance attribute `age` that can be different for each student.

Explain: Which parts are the class, objects, instance attributes, shared class attribute, and methods?

Task 2: `__init__` and `__str__`

Create a `Course` class.

1. Let `__init__` save the course title and number of weeks.
2. Write `__str__` so printing a course object shows a friendly description, such as its title and duration.
3. Create two course objects. Print each object directly.

Explain: When does Python call `__init__`? When does it call `__str__`?

Task 3: Encapsulation

Create a `Wallet` class.

1. Store the balance on the wallet object.
2. Add `add_money(amount)`. Only accept an amount greater than zero.
3. Add `spend_money(amount)`. Spend only when the amount is positive and the wallet has enough money.
4. Add `get_balance()` to return the current balance.
5. Try valid and invalid amounts and print the balance after each attempt.

Explain: What rule does the class enforce before changing its data? Why do we use methods to change the balance?

Task 4: Single inheritance

Create a parent class `Employee` with a constructor that stores a name and a `clock_in()` method. Create a child class `Intern` with a constructor that also stores a school. Use `super()` to pass the name to the parent constructor. Add a `study()` method.

Create an `Intern` object and call both methods.

Explain: Which method came from the parent class?

Task 5: Multilevel inheritance

Create this chain: `Company` -> `Department` -> `SupportTeam`.

1. Put a method in `Company`.
2. Create the two child classes.
3. Call the `Company` method using a `SupportTeam` object.

Explain: How does the method travel down the inheritance chain?

Task 6: Hierarchical inheritance

Create a parent class `Employee` and two child classes: `Developer` and `Designer`.

1. Put a shared method in `Employee`.
2. Create one object of each child class.
3. Call the shared method on both objects.

Explain: What do the two child classes share?

Task 7: Multiple inheritance

Create a `ReportWriter` class with a method for writing a report and a `FileSaver` class with a method for saving a file. Create a `ReportTool` class that inherits from both.

Create a `ReportTool` object and call both inherited methods.

Explain: How many parent classes does `ReportTool` have?

Task 8: Hybrid inheritance

Create a small class structure that combines inheritance patterns. For example, a `Device` parent can have `CameraDevice` and `PhoneDevice` children, and a `SmartDevice` can inherit from both children.

Add one method to `Device`, one method to each branch, and show which methods a `SmartDevice` object can use.

Explain: Which inheritance patterns did you combine? Keep the structure small and draw it with arrows in a comment.

Task 9: Abstraction

Use `ABC` and `abstractmethod` from Python's `abc` module.

1. Create an abstract class `Notification` with an abstract method `send(message)`.
2. Create `EmailNotification` and `TextNotification` child classes.
3. Implement `send()` differently in each child.
4. Create each child object and call its method.

Explain: Why can a child object be created after implementing `send()`, while an incomplete abstract class cannot be used directly?

Task 10: Polymorphism

Use the notification classes from Task 9, or create two report classes.

1. Make sure both object types have a method with the same name, such as `send()` or `show()`.
2. Write one function that accepts an object and calls that method.
3. Pass objects of both types to the same function.

Explain: Which call stays the same, and what changes between objects?

Task 11: Overloading in Python

Python does not let a class keep multiple methods with the same name and different parameter lists; a later definition replaces an earlier one. Practise the Python approach:

1. Create a `Bill` class with a `total()` method.
2. Let the method accept a required price and optional quantity and discount.
3. Call it with only the price, then with all values.

Extra: Create a `Points` class and define `__add__` so adding two `Points` objects returns a new `Points` object with the combined value.

Explain: How do default parameter values let one method support different calls? What does operator overloading change?

Task 12: Overriding

Create a parent class `Delivery` with an `estimate_minutes()` method. Create `BikeDelivery` and `VanDelivery` child classes that override the method with different simple time calculations.

Create one object of each type and call `estimate_minutes()` on both.

Explain: What method did the child classes replace? How is overriding related to polymorphism?

Task 13: `__name__` and `__main__`

Create two files in the same folder:

1. In `message_tools.py`, write a function called `show_message()` that prints a message. Add an `if __name__ == "__main__":` block that calls the function.
2. In `run_message.py`, import `message_tools` and call `show_message()`.
3. Run each file directly and compare what happens.

Explain: What value does `__name__` have when a file is run directly? What is it called when that file is imported? What is `__main__` in the `if` check?

Part 2 - Add conditions, loops, and functions

Use the OOP skills from Part 1. Each task introduces one programming tool at a time.

Task 14: Get input and create objects

Use the `Student` class from Task 1.

1. Ask for a student's name, age, and course.
2. Convert age from text to an integer.
3. Pass the answers into the class to create a student object.
4. Print the student's details using a method.

Explain: Why is `int()` needed for age? Which line creates the object?

Task 15: Add conditions to an object method

Update the `Wallet` from Task 3.

1. Reject zero or negative deposits.
2. Reject spending that is zero, negative, or greater than the balance.
3. Return a clear success or error message from each method.
4. Try one valid and one invalid example of each operation.

Task 16: Add a loop to process several inputs

Create a `GradeBook` class with a list of marks and an `add_mark(mark)` method.

1. Ask how many marks the user wants to enter.
2. Use a `for` loop to collect that many marks and add them to the object.
3. Use a condition to accept only marks from 0 through 100. Skip invalid marks and explain why they were rejected.
4. Print all accepted marks.

Task 17: Use a loop and condition to summarize data

Add methods to `GradeBook`:

1. `average()` should calculate and return the average mark. Handle the case where there are no marks.
2. `highest_mark()` should return the highest mark, or a helpful message if the list is empty.
3. `result()` should use a condition to return `Pass` when the average is at least 40 and `Needs improvement` otherwise.

Call each method and print its result.

Task 18: Use a separate function with objects

Create a `Product` class with a name, price, and stock count. Add methods to sell a quantity and restock a quantity. Use conditions to prevent selling more than the available stock or using negative quantities.

Create several products and store them in a list. Also make a dictionary that maps each product ID to its `Product` object. Then write regular functions outside the class:

1. `print_product_details(product)` should accept a `Product` object and print its details.
2. `find_product_by_name(products, name)` should loop through a list of product objects and return the matching object, or `None` if there is no match.

Call both functions and try valid and invalid sales. Use the list to show every product and the dictionary to find one product quickly by ID.

Explain: Which actions belong inside the object? Which jobs are handled by the separate functions?

Task 19: Input-driven menu loop

Use the `Product` class from Task 18.

1. Display a menu in a loop: show products, sell a product, restock a product, or quit.
2. Use conditions (`if` / `elif` / `else`) to handle the selected option.
3. Use functions to keep each menu action clear, such as `show_products()` and `sell_product()`.
4. Stop the loop when the user chooses quit.
5. Show a helpful message for an unknown menu choice or product name.

Task 20: Practise function arguments with objects

Create a `Workshop` class with a name, topic, and duration, then create a workshop object. Write regular functions that work with workshop details.

1. Write `print_workshop(name, topic="Python basics")`. Call it with and without the optional topic.
2. Call the function using positional arguments, then using keyword arguments.
3. Write `add_scores(*scores)` that accepts any number of scores and returns their total. Try calls with different numbers of scores.
4. Write `show_details(**details)` that accepts named details and prints each key and value. Pass the workshop object's name, topic, and duration as keyword details.

Explain: What kind of values go into `*scores`? What kind go into `**details`?

Task 21: Handle invalid input

Ask the user for a student's age or mark and safely convert it to an integer.

1. Put the conversion inside `try`.
2. Catch `ValueError` if the user types letters instead of a number.
3. Use `else` to process the value only when conversion works.
4. Use `finally` to print that the input attempt is complete.
5. Use a condition to check the range (age must be positive, or marks must be 0-100).

Task 22: Keep asking until input is valid

Use a `while` loop to ask for a mark until the user enters a number from 0 through 100.

1. Use `try` / `except ValueError` so text input does not stop the program.
2. Use a condition to reject out-of-range marks.
3. Use `break` after a valid mark is entered.
4. Create a `Student` object using the accepted mark.

Explain: Why does the loop continue for invalid input and stop for valid input?

Optional function extensions

- Write a recursive function that counts down from a positive number to zero. Identify the case that stops the recursion.
- Use a short `lambda` function to sort a list of `Student` objects by their marks or names.
- Avoid a global variable for the student list; pass the list to functions that need it.

Part 3 - Exception-handling practice

These tasks practise the exception-handling topics from Module 5 while working with objects.

Task 23: Validate data when creating an object

Update `Student` so its mark must be between 0 and 100.

1. Raise `ValueError` with a clear message when the mark is outside the range.
2. Try creating one student with a valid mark and one with an invalid mark.
3. Catch `ValueError` around object creation so the program can show the message and continue.

Task 24: Safely update an object in the menu

Update the product menu from Task 19.

1. Catch `ValueError` if the menu choice or quantity cannot be converted to a number.
2. Catch `KeyError` if the user selects a product ID that is not in the product dictionary.
3. Use `else` for the successful purchase update.
4. Use `finally` to print that the purchase attempt has finished.
5. Make sure a failed purchase does not change stock or the cart total.

Task 25: Process records when some data is invalid

Create a list of student data with marks saved as text. Include one non-number and one mark outside 0-100.

1. Loop through the records and create a `Student` object for each valid record.
2. Use `try` / `except` inside the loop to handle each bad record separately.
3. Print each valid student's result and a clear message for each invalid record.
4. At the end, print counts of accepted and rejected records.

Explain: Why can the program continue to the next record after one record causes an error?

Part 4 - Final project: Training Centre Manager

Build a small console program for a training centre. This project combines the topics from the lessons. Work through the requirements in order. Keep information in Python objects, lists, and dictionaries while the program runs; do not use a database or files.

A. Create the classes

1. Create an abstract parent class `Person` with an abstract method `get_summary()`.
2. Create `Student` and `Instructor` child classes. Both must implement `get_summary()` in their own way.
3. Use `super().__init__(...)` to reuse the parent's setup if you add shared information to `Person`.
4. Each student should store a name, student ID, course, and a list of marks.
5. Each instructor should store a name and a subject.
6. Give the classes a useful `__str__` method so printing an object shows a readable summary.
7. Add a class attribute for the training centre name that all people share.

B. Apply encapsulation

1. Store the student's marks on the student object.
2. Add a method to add a mark. Accept only values from 0 through 100.
3. Add a method to show the marks. Use the methods to update and read the marks rather than changing them directly in the menu code.

C. Add student and instructor behavior

For `Student`, add methods to:

1. Add a mark after validating it.
2. Calculate the average, handling a student with no marks.
3. Return a result using a condition: average of at least 40 passes; otherwise the student needs improvement.

For `Instructor`, add a method that returns the subject they teach.

D. Use polymorphism and overriding

1. Write a function `print_person_summary(person)` that calls `get_summary()` without checking whether the object is a student or instructor.
2. Pass both types of objects to that same function.
3. Explain how each class provides its own version of the shared method.

E. Build the menu with functions, conditions, and loops

Write a `main()` function that repeatedly displays this menu:

1. Add a student.
2. Add an instructor.
3. Add a mark to a student.
4. Show available courses.
5. Show all people.
6. Show a student's average and result.
7. Quit.

Requirements:

- Store people in a list of objects.
- Store available course IDs and course names in a dictionary. Enrol students in a course selected from this dictionary.
- Use a `while` loop so the menu appears again until the user quits.
- Use `if` / `elif` / `else` to respond to menu choices.
- Use `for` loops to display all people and search for a student by ID.
- Put repeated jobs in separate functions, such as `find_student()`, `add_student()`, and `show_people()`.
- Handle a missing student ID, invalid mark, empty list, and unknown menu choice with a clear message.
- Use `try` / `except ValueError` when converting menu choices, ages, IDs, or marks from text to numbers. A bad answer must not stop the program.
- Catch `KeyError` when a selected course ID is not in the course dictionary.
- Raise `ValueError` from a class method when a mark is outside 0-100; catch it in the menu and show a helpful message.
- Use `else` for an action that should happen only after valid input, and `finally` to report that the attempt is complete.
- Give `add_student()` a default course argument. Write a helper that accepts `*marks` to calculate a total or average, and a `show_person_details(**details)` helper to display named details.
- Keep input/output in the menu functions; keep each class responsible for its own data and behavior.

F. Use the `__main__` check

At the bottom of the project file, call `main()` only inside:

```
if __name__ == "__main__":
    main()
```

This means the menu starts when the project file is run directly. If another Python file imports the project, the menu does not start automatically.

Final project checklist

- [ ] I created classes and multiple objects.
- [ ] I used `__init__` to set up objects and `self` to store each object's own information.
- [ ] I used methods to keep data changes controlled (encapsulation).
- [ ] I used inheritance, including an abstract parent class and child classes.
- [ ] I showed overriding and polymorphism with a shared method.
- [ ] I used default arguments, `*args`, and `**kwargs` in helper functions.
- [ ] I practised Python-style optional arguments and operator overloading in Part 1 (Python does not support traditional same-name method overloading).
- [ ] I used `__str__` to make objects readable when printed.
- [ ] I used conditions, `for` and `while` loops, and separate functions.
- [ ] I used a list of objects and a dictionary for course lookup.
- [ ] I handled invalid input with specific exceptions and practised `raise`, `else`, and `finally`.
- [ ] I used the `if __name__ == "__main__":` check.
- [ ] I tested ordinary input and edge cases such as empty lists and invalid values.

Reflection questions

1. What is the difference between a class and an object?
2. What is the difference between an instance attribute (`self.name`) and a class attribute (`school_name`)?
3. How are inheritance and polymorphism different?
4. What is the difference between overriding and Python-style overloading with optional arguments?
5. What do `__str__`, `__name__`, and `__main__` each do?
6. When would you use a list, and when would you use a dictionary in this project?
7. Which part of your project was easiest to understand? Which would you improve next?